Portfolio123 Language
Acronyms and Functions

By

Marc Gerstein
February 2015

Once upon a time, you couldn't do anything in Portfolio123 unless you specifically phrased every screening/buy/sell rule or rank factor in Portfolio123 syntax. For example, you could not input anything like this:

Price-Earnings using Reported EPS – Less Than 20

You had no choice but to phrase that rule like this:

PEExclXorTTM<20

Thanks to the subsequent introduction of the Rules Wizard followed by the addition of plain English labels and Wizard-like input features for many free-form screening factors, the platform has become much more accessible to many more users.

That said, there is necessarily a tradeoff between simplicity and power. While strict adherence to plain English and Wizard-like inputting gives you far more modelling power than can be imagined by users of most other platforms, the things you can accomplish on Portfolio123 increase exponentially if you are willing to tolerate the learning-curve inherent understanding Portfolio123 acronyms and functions.

The "bad" news is that there really is a learning curve, a need for more initial effort. The "good" news, however, is that when it comes to the learning curve, its bark is much worse than its bite. Once you dive in, you'll see that this is much more manageable than what first impression might suggest. We have no doubt that the potential returns on investment of effort are compelling. Moreover, this is especially important for those who want to use technical and statistical factors in their screens since the Wizard is limited in this area. And it's vital for ETF and CEF investors since the Wizard applies to stocks only.

Acronyms

Surprisingly Intuitive

We understand that acronyms could look scary to a user who suddenly encounters them en masse in a screen or ranking system with many factors of formulas, especially if the model was designed by Portfoli123 or another user. But it's quite another matter to encounter them in a section of the interface that is being seen because you clicked on an item that relates to something you're looking for.

Suppose, for example, you want to work with a PEG (PE-to-Growth) Ratio. That wouldn't get you 100% of the way home. If you are a value-oriented user, you may not recognize, on first glance, the specifics of PEGLT or PEGIncXor. But having searched for a value-related item, you would likely recognize right away that you're looking at various kinds of PEG ratios. To one who is that well primed, the descriptions would register quickly and easily.

Here's the full description of PEGLT:

> Price to Earnings growth ratio is a simple valuation rule of thumb, it states: *"The p/e ratio of any company that's fairly priced will equal its growth rate."* A value less than 1.0 might reflect potentially undervalued companies and potentially overvalued companies when the value is greater than 1.0.
>
> This version, the PEGLT, is the more traditional version of PEG that is calculated by dividing the current year's projected PE (ProjPECurrFY) by the Long Term Growth Rate Estimate (LTGrthRtMean). See www.fool.com for a more detailed explanation. During the 4th Quarter, the next year's projected PE is used (ProjPENextFY).

Here's the definition of the other version of PEG we mentioned; PEGIncXor:

> This version, the PEGInclXor, is a variation of PEG that uses EPSPInclXorTTM instead of EPSPExclXorTTM.

EPSPInclXorTTM and EPSPExclXorTTM are both clickable, so if you want, you can go further and retrieve definitions of those terms. But you probably don't need to do that. You already can see you're dealing with some sort of definition of EPS. It's easy to recognize that ExclXor means excluding extraordinary items and that InclXor means including extraordinary items. And very early in your Portfolio123 endeavors the abbreviation TTM will become ingrained in your mind to the point where you instantaneously recognize it as referring to the trailing 12 month period.

Once you see this, you're likely to retain the definitions since, as noted, they derive closely from concepts that are already top of mind for you.

Simple Abbreviations you'll see Again and Again

Portfolio123 is not about memorization, a point we'll strongly reiterate when we address functions. But still, there is a core set of basics – the time period identifications – that are so recognizable, there's probably no way you could prevent yourself from quickly memorizing them even if you tried. Assume, for purposes of illustration, that the most recently reported quarter ended 9/30/07.

- TTM
  This means trailing 12 months (which can alternately be stated as "latest 12 months"). In this example, it would include the four quarters ending 9/30/07. This is important. The latest annual period, the one to which many investors and commentators often refer, ended 12/31/06 and would, by now, be pretty stale. When using factors from the income statement or statement of cash flows, try to get in the habit of using TTM rather than annual numbers.

- PTM
  This means prior 12 months. In this example, it would include the four quarters ending 9/30/06.

- Q
  This means latest quarter. In this example, it would refer to the quarter that ended 9/30/07. As with TTM, this will often be fresher than the latest annual period, which ended 12/31/06. Keep this in mind as you screen for balance sheet items.

- A
  This means annual, the latest annual period; in this example, the year that ended 12/31/06.

- PYQ
  This means prior-year-quarter comparison. In this example, we'd be comparing the quarter ended 9/30/07 with the one that ended 9/30/06. It's often referred to in conversation as the latest quarterly "year-to-year" or "year-over-year" comparison.

- PQ
  This means prior-quarter comparison. In this example, we'd be comparing the quarter ended 9/30/07 with the one that ended 6/30/07. It's often referred to in conversation as the latest "consecutive quarter" or "quarter to quarter" or "quarter on quarter" comparison.

- 3Y, 5Y and 10Y mean three-year, five-year and 10-year respectively.

Rapid Recognition

Ultimately, most of what you need to help you decipher the acronyms lies not in the simple descriptions, not in the full descriptions, not in any other kind of glossary or Help material, but in what you already know and the expectations you already have, when you click the mouse.

As an investor, let's assume you want to work with EPS Growth. To find your way to the factor(s) you need, you will have either (i) searched for "growth," (ii) searched for "eps," (iii) searched for "eps growth", (iv) or surfed your way through the factors menu by clicking on the sequence: FUNDAMENTALS>>RATIOS & STATISTICS>>GROWTH RATES.

Whichever route you chose, you wound up with your cursor on a plain-English prompt that says "Earnings Per Share (EPS) Growth." As a result of all this, you see the following set of factors:

EPS%ChgPQ
EPS%ChgPYQ
EPS%ChgTTM
EPS3YCG%
EPS5YCG%
EPS10YCG%

Yes, you can use descriptions to look up any or all of them. But do you really need to do that?

Now consider the process of getting to the one you want. Your eye will get to the desired item much quicker than if you had to look through a list consisting entirely of the plain-English equivalents:

EPS Percent Change, Most Recent Quarter vs. Prior Quarter (%)
EPS Percent Change, Most Recent Quarter vs. Quarter 1 Year Ago (%)
EPS Percent Change, TTM Over TTM (%)
EPS Growth Rate, 3 Years (%)
Earnings Per Share, 5 Year Growth Rate (%)
Earnings Per Share, 10 Year Growth Rate (%)

(By the way, it's not as if we withhold the plain-English labels from you. If you click on an acronym, the plain-English will immediately become visible below the factor menu.)

Your eye will instinctively eliminate the letters EPS since you already know that everything relates to that. And it'll take barely a nanosecond for your eye to

recognize that all of these items, whether labeled %Chg or CG% are dealing with growth rates. So again, these will be quickly dismissed. Now, all you would be focusing on are the time-period identifications. Once you familiarize yourself with those early in your work with Portfoio123, you would pick out the acronym you want and wind up doing so far quicker than you could click on and read a plain-English translation or a description.

Ultimately, the best way to get comfortable with the available factors, and the acronyms used to identify them, is to just browse. Most you will recognize. Sometimes you will need the short descriptions. And on occasion, you'll want to examine the full descriptions.

But far more often than not, you'll be very pleasantly surprised at how easy all this really is, especially when you approach the task with awareness of your background as an investor and your pre-existing sense of what the Category names mean to you.

Portfolio123 Functions

This is going to be more intricate than what we've seen thus far. But it, too, should be quite manageable once you get used to the idea of specifying parameters for a function – whatever that means (don't worry, you'll find out very soon).

Before we start, it's important to take note of the descriptions that are embedded in the interface. When we discussed acronyms above, it was suggested that Descriptions are often not critical. Functions are different. The Full Descriptions serve as vital references. Look at them, and do so often. They're direct and to the point, so this won't be time-consuming. And *no prizes are awarded to those who can memorize and retain all the information on what parameter goes where for the various functions*. The information will likely sink in for functions you use often. But otherwise, it's not worth the trouble to memorize.

We'll start with a general discussion of what a function is and how it works. This is where you'll learn all about parameters.

Then, we'll demonstrate some different kinds of functions. That will show you what function-based rules look like in a screen and teach you how to use the descriptions as a guide. All of these will be equally applicable to stocks, ETFs and CEFs.

What Functions are and how they Work

Essentially, a function is a mini program. It consists of two parts; (i) the logic, and (ii) the parameters.

- The logic, which is used over and over again, consists of directions telling Portfolio123 how to calculate the output you need. This is hidden from your view. All you see is a label, the name of the function.

- Logic is important, but it alone can't help you screen. You need to apply the logic to a set of specific inputs. These are the parameters.

Let's consider something very simple. Suppose we want to compute the average of three numbers. We don't know ahead of time what the numbers will be, so for programming purposes, we'll use placeholder labels: x, y and z.

The logic can be expressed as follows:

(x + y + z) / 3

Or, you might have a function that looks like this:

Avg(x,y,z)

You, the user, must tell the function what numbers should be swapped for x, y and z. So you might specify this:

Avg(4,-12,24)

That function would be executed by Portfolio123 by doing this:

(4 – 12 + 24) / 3 = 5.33

Or, you might do this:

Avg(825,38,514)

The result would be this:

(825 + 38 + 514) / 3 = 459

Look at what's happening. You can compute average after average without ever having to specify the logic. The function already knows what to do. Your role is to replace the placeholders with your input, the numbers that need to be averaged.

That's a very easy function. Now let's look at a more useful variation, an Avg that can work with 2 to 20 numbers without knowing in advance exactly how many numbers there would be.

A Portfolio123 formal description for such a function would look like this:

> Avg(x1,x2[,..,x20]): Returns the average value in list. Up to 20 parameters are allowed

The description is telling you to activate the Avg function by typing (or pointing to) Avg and specifying at least two and up to 20 parameters, and that the parameters you supply should be enclosed in parentheses and separated by commas. X1 and x2 are placeholders for the first two parameters; these are mandatory.

Everything in the brackets, [,...,x20] in this case, additional parameters up to 20 of them, is optional. Don't worry if you don't recognize the [,...,x20] notation. The accompanying text will tell you what's needed, as was the case with a very similar Avg function included among those offered by Portfolio123.

The logic of the function presented here would, more or less, be as follows:

- Count up the number of items supplied by the user in the parentheses
- Let n = the number of items
    - If n <2, stop and report an error message
    - If n > 20, stop and report an error message
    - If n >= 2 and n <= 20, continue working
- Compute the sum of all the items the user supplied
- Divide the sum by n
- Report the result

Suppose you supplied this:

Avg(28,4)

The function would do this:

(28 + 4) / 2 = 16

Now, suppose you supplied this:

Avg(35,16,212,8,93,4)

The function would do this:

(35 + 16 + 212 + 8 + 93 + 4) / 6 = 61.33

As you can see, when you work with functions, you don't need to worry about the details of the logic. That's already been programmed and its part and parcel of the platform. Once you've identified the function you need, all you have to do is supply the parameters, making sure to place each number in the correct position.

Now, let's try this:

Avg(13, 45, 12, 67)

Simple:

(13+45+12 +67)/4 = 34.25

Now, let's try this:

Avg(13, 45, 12 67)

Now you get an error. The problem isn't the number of spaces between 13 and 45. You can have as many of those as you want. But even one space is too many if you don't use a comma to separate the parameters, as is the case with 12 and 67. Now, consider this:

Avg(13, 45, 1267)

This is actually worse. In the above example, you saw an error message telling you “Invalid criteria in Rule 1. Error near '67': Missing operator or comma.” Now, however, the function computes “cleanly,” but not the way you wanted it to compute. You get this:

(13+45+1267)/3 = 441.67

Because the inputs 12 and 67 were not separated by a comma, Portfolio123 could not read them as two separate numeric parameters. Worse, because the inputs 12 and 67 were not even separated by a space, Portfolio123 was unable to recognize an error and had no choice but to read this as a single four-digit number: 1267.

The moral of the story: Make sure your parameters are enclosed within parentheses and separated by commas!

In the above examples, sequence was irrelevant: Avg(5,10) produces the same result as Avg(10,5). But often, you’ll be using functions for which sequence does make a difference. An example is simple moving average, SMA: SMA(5,10) is not the same as SMA(10,5). And speaking of SMA, let’s make that the first in a series of examples which will be divided into two categories, technical stock analysis (also applicable to ETFs and CEFs), and fundamental stock analysis.

By the way, don’t get too hung up on the distinction between technical and fundamental analysis; no matter how committed to fundamentals-only you might be, review the technical-analysis examples. What you’ll learn about functions in general will be invaluable. And you really don’t want to miss the last technical example, which, if you’ve never used a platform like Portfolio123, will rock your world!

Technical Analysis Functions

Simple Moving Average

Find sma either by searching or using the menu tree: Go to SCREEN >> NEW STOCK SCREEN. Click on TECHNICAL – INDICATORS & PRICE >> MOVING AVERAGE >> Simple Moving Average.

The description tells us the syntax is SMA(bars [,offset,series]).

Based on what we already learned, we know that “bars” is a placeholder for a number. More specifically . . .

- Bars is the number of trading days over which the moving average should be computed. It has to be a number between 2 and 500.

The brackets tell us the next two parameters are optional.

- Offset is a days-ago number. An offset of 10 means we'd calculate what the 50-day moving average as it was 10 trading days ago. Offset can be between zero and 100.

  If we leave this blank, the function will assume it should be zero. In other words, there would be no offset and we'd compute the current moving average.

- Series answers the question: Compute a moving average of what?

  If we leave the Series parameter blank, the function will assume we want to calculate the moving average of closing prices. This information is contained in the Full Description, where we also see a list of alternative series we could use. If we're interested in market timing, we might use #Bench, which refers to the market benchmark that has been selected in the screener's Main Settings or under the General tab in Simulation/Portfolio (the default is the S&P 500).

Using this information, here are some possible ways we could use SMA (or sma or sMA or Sma; Portfolio123 is not case sensitive).

- SMA(50) > SMA(200)
  The 50-day moving average is above the 200-day moving average.

- SMA(50) > SMA(50,20)
  The latest 50-day moving average is above the level at which the 50-day moving average stood 20 days ago.

- SMA(50,0,#Bench)>SMA(200,0,#Bench)
  The 50-day moving average of the market is above its 200-day moving average. If this condition is violated, no stocks will pass the screen, meaning we'd in essence, be going all cash; hence the market timing element.

Notice in the last example, how use of the third parameter, the series, forced us to also use the second parameter even though it was originally optional. Notice, too, that parameters need not always be stated as numbers. The descriptions will tell you what entries are permissible for each parameter.

For a second parameter, the SMA logic expects to see nothing at all, or a number. So if we use SMA(50,#Bench), we'd get an error message because the logic would not be able to cope with seeing #Bench where it expects a number. So even if we don't want an offset, we must plug in a zero in order to push #Bench to the place where it belongs, the third position.

IMPORTANT: It is critical that each parameter be placed in the correct position, even if it means using zero for some optional parameters that we'd like to skip.

Pause here and review SMA. Let it sink in. Although it's not the most elaborate function, it's effective as an illustration given its use of optional parameters one of which involves specification of a series. Once you feel fully comfortable with SMA, you should find most other functions quite easy.

When you feel ready to go on, we'll tackle GapUp, which is interesting in that unlike SMA, we're not asking the logic to give us numeric output. Instead, we're looking for a yes or no; either the pattern we specify happened or it didn't happen.

Gap Up

This is where investors look for stocks that experienced a significant upward gap between the closing price on day one and the open price on day two. On Portfolio123, those seeking to implement this strategy can use the GapUp function. (There is also a GapDown function for those who are interested in the opposite approach.) These can be found by searching, or through the menu tree: TECHNICAL – INDICATORS & PRICE >> PATTERN >> GapUp.

As with other functions, you do not need to program the details of an upward gap. All the necessary logic is stored inside the function and will execute when Portfolio123 finds the function in a rule or rank factor. You only need to supply the parameters, the important numbers that vary from investor to investor based on individual preference and even with the same investor at different points in time.

Let's assume you want to look for a stock that experienced a gap up within the last five trading days.

- By thinking in terms of five trading days, we have already established one parameter

- Another parameter is the size of the gap. Will a 1% jump interest us? Most who use a strategy like this won't care about a move that small. Let's say we are interested in up gaps of at least 10%. That is another parameter.

This alone is enough to implement the strategy. Here's the function description:

GapUp(GapPct,VolPct,bars,offset)

Notice that the function requires four parameters. We have already identified the numbers we'll use for two of them: we decided bars will be set at 5, and that GapPct will be set at 10. We're not using the other parameters. We don't care

about volume and we want the function to execute as of now, so we aren't using an offset.

So it might seem that we should articulate the function as follows:

GapUp(10,5)

That will not work!

Look again at the function description. None of the parameters are identified as being optional, as would be the case if the description looked like this: GapUp(GapPct,bars[,VolPct,offset]). But that's not what we see. Nothing is enclosed in brackets. That means everything is mandatory. If we don't want to assign values, we have to plug in zeros. So let's fix it.

GapUp(10,5,0,0)

Again, this is not correct.

It's not enough to have the correct number of parameters. Each one must be in the proper position.

The function we wrote starts reasonably; Portfolio123 will look for stocks that experienced gaps of at least 10%. It will refine the search by limiting consideration to those whose volume on the day of the gap was at least 5% above the average experienced over the number of days we set with the bars parameter. So far, so good . . . sort of. It is possible that we want 10 and 5 in the GapPct and VolPct positions, so this won't precipitate an error. But it's not what we chose (we wanted 5 for bars, not for VolPct) so the result will be wrong. Next, the function will encounter a zero when it evaluates bars. That's illogical. We need a number for bars.

All this is meant to drive home the importance of function descriptions, how these are vital references that should be used regularly. Again, ***when it comes to functions, no prizes are awarded for memorization or successful guessing.***

Let's now write the function correctly, with all required parameters in place, in their proper positions, and with zero if necessary to populate a parameter we'd rather not use.

GapUp(10,0,5,0)

That is correct. It will show us the stocks we want to see.

Many will not want zero in the second position, the VolPct parameter. They'd rather restrict consideration to gaps that were accompanied by noteworthy

volume spikes. Let's assume we decide to do that; we'll insist that gap-up volume be at least 600% above the average of the last 5 trading days. If we choose to do this, our function would look like this:

GapUp(10,600,5,0)

So now that we have this function, what should we do with it? How can we get Portfolio123 to use it?

As noted, this is not like SMA, which calculates a number. This function looks for a yes or no answer. Either a stock experienced a gap up under the conditions we articulated, or it didn't; yes or no. In computer-talk, that translates to true or false. Accordingly, we can create a rule that looks like this:

GapUp(10,500,5,0) = true

It would also be ok to leave the function in the rule by itself. All factors or functions that are sitting alone will be evaluated by Portfolio123 as if the "= true" were also there. So the following alternative rules would produce identical results:

GapUp(10,500,5,0)
GapUp(10,500,5,0) = true

(NOTE: By the way, the implied "=ture" feature exists for every Portfolio123 factor or function. Start a new screen and for the first rule, just type the word price. Click on Totals to see how many stocks pass the screen. You'll notice that every stock in the database qualifies. By typing price alone you implied price=true, which meant you searched for every stock that has a stock price. Since all stocks in the database have prices, all pass.)

As a bonus, here's one more computer=-speak tidbit. You could also choose to use 1/0 instead of true/false. So there are actually three ways you could specify the GapUp rule:

GapUp(10,500,5,0)
GapUp(10,500,5,0) = true
GapUp(10,500,5,0) = 1

Those interested in using technical indicators should be good to go right now. There are many more such functions that have not been demonstrated here. But now that you understand what functions are and how they work, and most importantly, how to specify parameters, you should be able to work with the descriptions (if a Full Description is available, use that) to incorporate any of the other technical indicators into your models. But don't stop reading here. The next two examples address items that some may not regard as strictly being the

province of technical analysis but which are nonetheless in that ballpark and often of interest to technically-inclined investors.

Statistical Functions - Sharpe

The functions included among those in the TECHNICAL – INDICATORS AND PRICE >> VOLATILITY category include several functions that could just as easily be thought of as being Statistical. They can definitely be of use to equity investors, but are especially worth noting for use with ETFs and CEFs. These functions, which should be familiar to those who are acquainted with MPT (Modern Portfolio Theory), are:

- Average Percent Change, or PctAvg (the average of the percentage moves of the closing prices)

- Standard Deviation, or PctDev (the standard deviation of the percentage moves of the closing prices)

- Sharpe Ratio (a Sharpe-like ratio, return divided by standard deviation ; unlike the normal Sharpe ratio, it is not adjusted for the risk-free return)

- Sortino Ratio ((a Sortino-like ratio, return divided by downside deviation; unlike the normal Sortino ratio, it is not adjusted for the risk-free return)

All have thorough descriptions to guide you through the parameters. We'll demonstrate Sharpe here.

The Sharpe function balances return against risk: its average price percent change divided by the standard deviation of the percent changes. (Unlike the textbook version of Sharpe, we don't subtract the risk-free rate from return.) Details of our calculation can be seen in an Excel spreadsheet you can access by clicking on a link in the Full Description.

This function (as well as Sortino, a variation that defines risk in terms of downside deviation) comes in two easy flavors, 1Y and 2Y versions (one-year and two years respectively based on weekly returns; no parameters need be input). The Sharpe we'll demonstrate here is the flexible one, the one that allows you to specify parameters. Here's the formal description:

Sharpe(range[,bars,offset])

- Range, the only mandatory parameter, refers to the number of bars in the overall period to be studied. If you want to use one year, the parameter should be set at 250, the approximate number of trading days used in a year. This number can go up to 502.

- Bars, the first optional parameter, refers to the number of bars that should be used to calculate return. If you omit the parameter, the function will assume you want 5, which usually amounts to a trading week. This number can go up to 60.

    This requires some thought. We're allowing you input two different numbers as bars. The best way to visualize the difference is to consider some examples.

    - Sharpe(250,5) will examine weekly returns logged over the course of a one-year study period. Since 5 is the default value for the second parameter, you'd have achieved the same result had you used Sharpe(250).

    - Sharpe(250,1) will examine daily returns logged over the course of a one-year study period

    - Sharpe(60,1) will examine daily returns logged over the course of a sixty-day study period

    - Sharpe(500,20) will examine twenty-day returns logged over the course of an approximately two-year period

    This should suffice to show you how it works.

- Offset tells you how many days (bars) back you should count before starting the calculation

    - Sharpe(60,1,20) will examine daily returns logged over the course of a sixty-day study period the most recent day having occurred about one trading month (20 days) ago

    - Sharpe(250,5,125) will examine weekly (5-day) returns over the course of an approximate trading year that ended 125 days ago. In contrast to what we saw with Sharpe(250,5), the second parameter is not optional here. Once we decide to make use of the optional third parameter, the second parameter becomes mandatory, even if we just want to repeat the default value – this is necessary to make sure 125 winds up in the third position.

    - Use of offset can support some interesting ideas. For example, the rule Sharpe(125,5,0)>Sharpe(125,5,60) will identify situations where the Sharp ratio has been improving over the course of the last three months. This could have been written as Sharpe(125,5)>Sharpe(125,5,60). I didn't really need to specify the third parameter in the first

expression, since it is zero. I chose to do so for my own convenience.

Once you get comfortable using Sharpe, you should easily be able to work with descriptions to use the other statistical functions.

Now, let's move on to another collection of functions that seem deceptively simple, but actually, pack quite a punch.

Price & Volume Functions - Close

Start a new screen. We'll work with Close, which can be found by searching or via the menu tree: TECHNICAL – INDICATORS & PRICE >> PRICE & VOLUME >> Price OHLC. OHLC stands for Open, High, Low, Close.

As noted, we'll work with Close. The idea is simple enough; it refers to closing stock price, and it has one mandatory parameter, BarsAgo, which tells us how many bars we should count back. The most recent closing price would be Close(0). The closing price two days ago is Close(2).

Here's how you could compute a one trading week (five day) percent price change:

- close(0)/close(5) . . . if you're willing to have your answer expressed as .993, 1.025, etc.

- (close(0)/close(5))-1 . . . if you want the decimal equivalent to a percent, such as -.007, .025, etc.

- ((close(0)/ close(5))-1)*100 . . . if you want a percent value, such as -.7, 2.5, etc.

The really interesting question is: Closing price of what?

The default answer is the stock price. But you can add a second optional parameter, a series, to get the closing value of something else, such as the benchmark of your choice (the default being the S&P 500), the 10-year treasury yield, the consensus current-year EPS estimate for the S&P 500, and so forth. The Full Description lists the possibilities for the series parameter. Here are some sample rules.

- Close(0,#TNX) <= Close(10,#TNX) + .25
  The ten year treasury yield did not rise more than 25 basis points over the past 10 trading days

- Close(0,#SPEPSCY)>=Close(20,#SPEPSCY)

Consensus current-year estimates for S&P 500 did not, on the whole decline in the past 20 trading days

Both of the above samples would be market timing rules. If the conditions fail, no stocks would pass your screen, meaning you'd be entirely in cash.

Now, let's have some fun – really!

Flexing Our Screening Muscles

Suppose you don't want an all-or-nothing approach to market timing. Perhaps you want to just restrict yourselves to large-cap or low-beta stocks when you're nervous about the market.

You would then use the Eval function, which brings if-then logic to Portfolio123. You can find Eval and its description by searching or surfing to the MISC>>UTILIYU category of functions. Here's the formal definition:

Eval(condition,expr1,expr2)

- Condition is just that, a condition, a statement that might be true or might be false
- expr1, or expression 1, is the item (a number or the result of a factor or formula) Portfolio123 should use if the condition is true
- expr2 or expression 2 is the item Portfolio123 should use if the condition is false.

Phew, that's a mouthful. Take some time to digest it. When you're ready brace for another whopper: You can use another function to specify the Condition parameter; yes, Portfolio123 allows you to use functions within functions. Now, take a look at this:

Eval(Close(0,#SPEPSCY)>=Close(20,#SPEPSCY),close(0)>=0,beta<=.9 or MktCap>=5000)

To a functions novice, this looks like a monstrosity but it really isn't. We have all the tools we need to break it down.

- Let's start with the set of inside-the-expression functions, Close:
    - Close(0,#SPEPSCY) is the latest close for the series SPEPSCY, which is the consensus estimate of current-year EPS for the S&P 500
    - Close(20,#SPEPSCY) is 20-trading-day-ago close for the series SPEPSCY, which, again, is the consensus estimate of current-year EPS for the S&P 500

- So now, we know what the condition is:
    - Close(0,#SPEPSCY)>=Close(20,#SPEPSCY),close(0)
    - The consensus current-year EPS estimate for the S&P 500 is equal to or above where it stood 20 trading days ago, or put another way
    - The consensus current-year EPS estimate for the S&P 500 did not decline in the last 20 trading days
    - That's always going to be true or false
- Let's now see what happens if the condition is true; i.e., if the estimate did not decline in the last 20 days; a neutral/benign scenario (whether that's bullish is a matter of opinion, but I think we can agree it's not bearish)
    - In this case, Portfolio123 would apply expr1, which is the rule close(0)>=0, which means the latest closing stock price is equal to or above zero
    - This winds up accepting/passing all stocks, since all have prices that are equal to or above zero
    - In other words, if the market looks no worse than neutral (as per our definition of it, in the condition), then nothing is eliminated. Portfolio123 moves and executes the remaining rules
- Now, let's see what happens if the condition is false; if the S&P 500 EPS estimate did decline within the past 20 days
    - In this case, Portfolio123 would apply expr2, which is a very different rule: beta<=.9 or MktCap>=5000
    - This is a screening rule that would cause Portfolio123 to limit consideration to large-cap stocks, defined in expr2 as those with market capitalizations of at least $5 billion or to less volatile stocks, which we define as those whose betas are less than or equal to 0.9.
- So in plain English, here's what this rule does:
    - It checks to see if the S&P 500 estimate fell within the past 20 days; if so, it limits consideration to stocks that are larger or demonstrably less volatile; if not (i.e. if the market looks no worse than neutral), Portfolio123 yawns, does nothing, and moves onto the next rule. Or put another way

- It takes the temperature of the market; if things look OK, no problem, move on to the rest of the screen; if we're seeing a red flag, we take protective measures by limiting consideration to "defensive" stocks

Yes, that's complicated. Yes, it's not a rule we can expect novices to write (although novices could certainly copy-and-paste it from this tutorial or could do likewise with similar rules posted in the Forum or included in a user-saved models that set visibility at "Public).

But seriously, that's one heck of a rule. Isn't it nice to be able to learn how to create it, or something similar, if you wish? You can't do it with Yahoo! You can't do it with AAII. You can't do it with Morningside. You can't do it with Zacks. And based on what I recall from the last time I saw FactSet (an elite very-high-priced institutional platform), you can't do it there either. This is an example of the power you unleash if you take the trouble to learn how to use Portfolio123 functions.

Fundamental Financial Statement Functions

Portfolio123 offers many ratios you can use to create fundamental screening and/or ranking rules (such as growth rates). Other times, however, you will want to use specific numbers (such as trailing 12 month revenues) whether on their own or to build ratios that are not pre-calculated by Portfolio123 (no matter how many ratios are offered, sooner or later you're going to find yourself needing something that's not available, such as year-to-year EPS growth rates for each of the past four quarters). You can create these and many other non-standard items right now using Portfolio123's Financial Statement functions.

Numbers vs. Ratios

Fundamental data used by Portfolio123 can be categorized in two ways.

First are raw data items we get from our data supplier, such as Revenues, EPS, Long-term Debt, etc.

Second is a set of ratios we calculate based on the raw items. Examples include EPS5YCGr% (the annualized five-year EPS growth rate), CurRatioQ (the current ratio in the latest quarter), or ROE%TTM (trailing 12 month return on equity).

If you review your screens and ranking systems, chances are you will have made heavy use of ratios. Perhaps you even use all ratios all the time. This would not be surprising. More often than not, individual numbers are of little interest.

For example, do we care deeply about whether EPS is greater than or less than,

say, $2.00? But we are likely to care very much if the $2.00 EPS figure is higher than a year-ago $1.50 figure, or less than a $2.50 year-earlier tally. We can address this without using ratios. We can simply require one EPS figure to be greater than another one. But that's not necessarily a great solution. There's a difference between a $2.00 EPS figure that is up from $1.50 a year earlier, versus a $2.00 tally that is above an earlier $1.97 base. We may want the former but not the latter. The way to distinguish is through a growth rate, which is a ratio.

An Example

Portfolio123 offers a strong collection of functions based on the financial statements. All are available on an annual, quarterly and trailing-12-month basis.

Let's take a basic example. Suppose you want a screening rule requiring each of the year-to-year EPS comparisons in the past year to be stronger than the one for the preceding quarter. Assuming we label the last-reported quarter as Quarter Zero (we don't count back any quarters; use the latest), here's how such a rule would look in plain English:

> *Year-to-year EPS growth rate in quarter zero (the last quarter) was above the rate achieved in quarter one (count back one quarter)*
>
> *and*
>
> *Year-to-year EPS growth rate in quarter one was above the rate achieved in quarter two*
>
> *and*
>
> *Year-to-year EPS growth rate in quarter two was above the rate achieved in quarter three*
>
> *and*
>
> *Year-to-year EPS growth rate in quarter three was above the rate achieved in quarter four*

Such a rule could appeal to an aggressive growth-oriented investor. But it cannot be implemented using the standard Portfolio123 ratio package. The rule can, however, be built using Portfolio123's Financial Statement functions.

How to Use Financial Statement Functions

The basic format is:

Item(offset, type[, NAHandling]).

Note that the last parameter, NAHandling, is optional.

Let's learn in a hands-on way.

First, we need to know where to find the line items. They are available through the free-form interface and, as we might expect, can be found via search or by starting in the top-level folder called FUNDAMENTALS. After opening that folder by clicking on the "+" sign, go to an open the second-level sub-folder called STATEMENMTS & FILINGS. You will then see four third-level sub-folders, two of which contain one more level of sub-folders

- INCOME STATEMENT
- BALANCE SHEET
  - ASSETS – CURRENT
  - ASSETS – NONCURRENT
  - LIABILITIES – CURRENT
  - LIABILITIES NONCURRENT
- CASH FLOW STATEMENT
  - OPERATING
  - INVESTING
  - FINANCING
  - SUMMARY
- SHARES

Open these folders and sub-folders and look around. All of the items should look familiar. Based on the above organization protocol, which matches that of the financial statements, you should easily be able to find the item you want. Once you do, we need to consider the three parameters.

Parameter 1 - Offset

This tells Portfolio123 how far back in time you want to go. The most recent period is always identified as 0 (zero), meaning you count back zero periods from the present.

Parameter 2 – Type

This tells Portfolio123 what kind of time period do we want the number to cover? Do we want Quarterly, Annual, or Trailing 12 Months? Time period identification should be easy to remember:

Qtr = Quarter
Ann = Annual
TTM = Trailing 12 Month

(NOTE: The interface is not case sensitive. You could just as easily type ANN, ttm, qTr, etc.)

Combining the Offset and Type Parameters

It's important to make sure you count back correctly, which is easy once you get comfortable with the way the numbering system works. As noted, the starting point is always the most recent period. You take the last period available and do not count back to any historical periods. Or to put it in computer-speak, you start with the latest available number and count back ZERO periods.

Let's make sure we really understand the offset. Suppose the most recent quarter is the one that ended 6/30/12. Here are the some offsets:

- QTR 0 ended 6/30/12
- QTR 1 ended 3/31/12
- QTR 2 ended 12/31/11
- QTR 3 ended 9/30/11
- Etc. up to QTR 20

Suppose the latest fiscal year, the annual period, ended 12/31/11. Here are some offsets:

- ANN 0 ended 12/31/11
- ANN 1 ended 12/31/10
- ANN 2 ended 12/31/09
- ANN 3 ended 12/31/08
- ANN 4 ended 12/31/07
- Etc. up to ANN 10

Now let's consider trailing 12 months (TTM):

- TTM 0 ended 6/30/12
- TTM 1 ended 3/31/12
- TTM 2 ended 12/31/11
- TTM 3 ended 9/30/11
- TTM 4 ended 6/30/11
- Etc. up to TTM 20

Notice what's happening with trailing 12 months. Some users erroneously believe that if they want to compare a TTM period ending 6/30/12 with a TTM period

ending 6/30/11, they think in terms of the annual structure and compare TTM 0 to TTM 1. That is not correct. We get a new TTM calculation after every quarter. Therefore, this comparison would be TTM 0 versus TTM 4.

Putting all this together, if you want the most recent quarterly EPS number, it would be EPSExclXor(0,qtr), assuming you want to exclude extraordinary items, as is usually the case. (You don't have to remember or type EPSExclXor; you can look for it in the reference menu and then point-and-click.)

Notice the how we identify which EPS figure you want (the function parameters). Start with the Item, which, as noted, you can get by clicking your mouse and then add parenthesis inside of which you'll place two parameters: the first is the offset and the second is the time-period-type designation: qtr, ann or ttm.

Suppose you want to use trailing-12 months EPS and count back two periods. (If the most recent report is for a quarter that ended June 30th, counting back two periods would bring you to a quarter that ended December 31st). It would be EPSExclXor(2,ttm).

All line item data goes back five years. That means . . .

The count-back number for a QTR item can be anything from 0 to 20

The count-back number for an ANN item can be anything from 0 to 10

The count-back number for a TTM item can be anything from 0 to 20

We're almost done. Remember mention of a third parameter, for NA handling. It's optional. Most users will be satisfied with the default and, hence, never use it. But for those who may want to vary, here goes . . . .

Parameter 3 - NAHandling

When companies report results after the end of a period, they don't always report all of their numbers at the same time. Often, they'll issue an earnings release that has a lot (much more now than in years past), but is not complete. The remaining numbers will be included in the formal SEC filings (such as 10-Q or 10-K), which are usually filed shortly after the earnings releases.

As a practical matter, the time lags between the earnings releases, the formal filings and inputs by the financial databases are shrinking. But they are not zero. That means there will be occasions when we are stuck with incomplete data.

> Solution 1 – Input and use as much data as we can as quickly as we can and cope with the gaps as best we can

Solution 2 – Hold off until all data is available thereby using perfectly complete datasets at all times.

Solution 2, wait until everything is complete, holds considerable "Ivory Tower" appeal and we allow users to select this option via the NA-Handling parameter. But most who are active in today's markets want as much data as they can get as quickly as they can get it. The default NA-Handling choice effectuates this most effectively.

The starting point here is to recognize that data that is missing items appear in the database as NA, which stands for Not Available. Here are the ways the NA-Handling parameter addresses this.

FALLBACK – This is the default choice. You don't have to spell it out; if you want to stay with it, you can leave the third parameter blank. FALLBACK means we compensate for missing items by falling back to the most recently available financial statement. So if, for example, an earnings release contains an income statement and balance sheet, payout ratio, which requires an item from the omitted cash-flow statement, will reflect a number that carries over from the last statement. The disadvantage is that the number is old. The benefit is that it will prevent the stock from automatically being excluded from a screen, as it would if a rule encountered an NA item. The good news, as noted, is that formal filings are being speeded up so this state of affairs tends to be very brief nowadays.

KEEPNA – This is the "Ivory Tower" solution. If up-to-date data is not available, then Portfolio123 reads the item as NA. The good aspect of this is that it is perfectly accurate. The negative is that many stocks may get kicked out of your models if, say, you rebalance a day or so before the complete data becomes available.

ZERONA – This is a variation on NA. You're staying accurate but by substituting a digit (zero) for NA, you're giving the stock a fighting chance of staying in a model.

A Simple Illustration

Suppose you're interested in the ratio of R&D (research and development) to sales. Here's what it would look like:

RandD(0,ttm)/Sales(0,ttm)

If you want the ratio to be better than it was in the previous trailing 12 month period, your rule would look like this:

RandD(0,ttm)/Sales(0,ttm) > RandD(4,ttm)/Sales(4,ttm)

A Comprehensive Illustration

Now that we understand how financial-statement functions work, let's go back and build that sequentially-improving EPS growth rule we presented above but were unable to implement using the pre-calculated ratios.

We're going to compute percentage growth rates like this: (X-Y)/abs(Y) where X is the more recent item and Y is the earlier one. It will enable us to comfortably handle situations where negative numbers may appear. Suppose, for example, EPS in the year-ago quarter was -.45 but is now 0.6. That's a nice improvement. We went from a noteworthy loss to a modest profit. But a conventional growth rate calculation, (X/Y)-1 would show it as a big decrease.

(X/Y)-1 = (.06/-.45)-1 = -113%

(X-Y)/abs(Y) = (.06 - -.45)/abs(-.45) = 113%

The second computation is the one that produces the more sensible answer, the one that shows nice improvement. This formula should be used any time you compute a percent change involving one or more numbers that might be negative. (If both numbers turn out positive, we'll get the same answer from both approaches.)

Using financial functions and the (X-Y)/abs(Y) growth-rate formula, here, now, is our sequentially-improving EPS growth-rate screen. (You could combine all this into a single rule through use of AND, but your screen is likely to be much more readable if you make each statement a separate rule.)

```
(EPSExclXor (0,qtr)- EPSExclXor (4,qtr))/abs(EPSExclXor(4,qtr))>(
EPSExclXor(1,qtr)- EPSExclXor(5,qtr))/abs(EPSExclXor(5,qtr))

(EPSExclXor (1,qtr)- EPSExclXor (5,qtr))/abs(EPSExclXor(5,qtr))>(
EPSExclXor(2,qtr)- EPSExclXor(6,qtr))/abs(EPSExclXor(6,qtr))

(EPSExclXor (2,qtr)- EPSExclXor (6,qtr))/abs(EPSExclXor(6,qtr))>(
EPSExclXor(3,qtr)- EPSExclXor(7,qtr))/abs(EPSExclXor(7,qtr))

(EPSExclXor (3,qtr)- EPSExclXor (7,qtr))/abs(EPSExclXor(7,qtr))>(
EPSExclXor(4,qtr)- EPSExclXor(8,qtr))/abs(EPSExclXor(8,qtr))
```

Try it out. Copy each rule into your clipboard and paste into a blank free-form line in the screener.

Ready to Go!

By the way, speaking of copy-and-paste with clipboards, this can be a very effective way to create many rules involving financial functions. These can get complex, not so much from the items per se, but from the need to make sure all your parentheses are correct. The most efficient way to handle this is to build your expression in Word or a text editor and then paste it into the screening interface.

Bottom line: The learning curve on the Financial Statement functions will usually be quick. The extent to which you will be able to effectively use them to create your own rules will probably depend on your ability to master the use of parentheses.